

Análise do Impacto de Variações de Prompting e Feedback Semântico no ARC-AGI-1: Fundamentação Estatística

Fabio Luiz da Costa Cieslak^{*1}, Izadora Candotti de Oliveira^{*1}, Matheus Machado^{*1}

Universidade Federal do Rio Grande do Sul¹

Abstract

O benchmark Abstraction and Reasoning Corpus (ARC-AGI-1) avalia a capacidade de raciocínio indutivo e generalização em sistemas de inteligência artificial. Investiga-se, neste trabalho, como variações estruturadas de prompt influenciam o desempenho de modelos de linguagem (LLMs) na síntese de programas executáveis em Python. Compara-se a geração direta em passo único (*1-Shot*) com ciclos iterativos de síntese indutiva guiada por contraexemplos (CEGIS), sob regras de restrição sintática e de elicitação de invariantes abstratos. Para quantificar os efeitos observados, formaliza-se um protocolo estatístico pareado fundamentado no teste exato de McNemar, reamostragem bootstrap com intervalos de confiança de 95% e uma taxonomia dos modos operacionais de falha (recuperação semântica, regressão, sobreajuste espúrio e teto de representação). Os experimentos conduzidos sobre nove modelos revelam que o feedback por contraexemplos proporciona ganhos expressivos de acurácia (com avanços de até +30%) sob estrita monotonicidade (zero regressões), estabelecendo um arcabouço rigoroso para discernir aprimoramentos reais do ruído estocástico.

Código e Dados — <https://github.com/Jubarte27/arc-agi-1>

1 Introdução

O Abstraction and Reasoning Corpus (ARC-AGI) (Chollet 2019) avalia a aquisição de habilidades e o raciocínio indutivo amplo em inteligência artificial, priorizando a capacidade de generalização frente a tarefas inéditas em detrimento da mera memorização. Cada problema é composto por um conjunto reduzido de 2 a 5 pares de matrizes de treino e uma ou mais grades de teste cujas saídas devem ser inferidas a partir da lógica subjacente observada.

Uma linha consolidada de investigação formula essa tarefa sob a ótica da síntese indutiva de programas (*Program Synthesis*) (Solar-Lezama 2008), na qual modelos de linguagem são instruídos a gerar uma função Python `transform(grid)`. A efetividade desse paradigma depende da arquitetura do prompt, dos limites impostos ao espaço de busca e da existência de mecanismos de retroalimentação baseados no resultado da execução.

^{*}These authors contributed equally.

O presente estudo investiga o impacto de variações sistêmicas de prompt no ARC-AGI-1, contrastando a síntese direta com ciclos de feedback semântico (CEGIS). Além de mensurar a eficácia dessas abordagens em nove modelos representativos, estabelece-se uma metodologia estatística pareada para orientar avaliações experimentais no benchmark e atenuar a ocorrência de falsos positivos decorrentes da variabilidade amostral.

O restante deste artigo organiza-se como segue: a Seção 2 detalha as variações de prompting; a Seção 3 formaliza a metodologia estatística pareada; a Seção 4 descreve o protocolo experimental e a infraestrutura; a Seção 5 discute os resultados empíricos; e as Seções 6 e 7 apresentam a discussão e as conclusões.

2 Variações de Prompting e Síntese de Programas

A síntese de soluções executáveis a partir de poucos pares demonstrativos impõe desafios acentuados de busca combinatorial e abstração geométrica. Com o intuito de investigar como a formulação do contexto modula o espaço de hipóteses explorado pelos modelos, examinam-se três regimes estruturados de interação.

2.1 Síntese Direta 1-Shot (Baseline)

Como ponto de partida comparativo (*baseline*), adota-se a síntese direta em passo único (*1-Shot*). Nessa modalidade, o modelo recebe a especificação formal da tarefa e a totalidade das matrizes de treino em formato JSON em uma única requisição, devendo produzir exclusivamente o bloco da função `transform(grid)`.

Para assegurar padronização de ambiente e reprodutibilidade estrita, o prompt veda a importação de módulos externos (`numpy`, `collections`, `math`), restringindo o código a tipos e funções nativas da linguagem (`list`, `dict`, `range`, `min`, `max`). Essa delimitação neutraliza divergências de dependências externas e induz o modelo a explicitar as transformações matriciais por meio de algoritmos determinísticos fundamentados na sintaxe básica.

2.2 Feedback Semântico via Contraexemplos (CEGIS)

A abordagem CEGIS (*Counterexample-Guided Inductive Synthesis*) estende o paradigma em direção a um ciclo iterativo de refinamento guiado por execução em sandbox isolada.

Caso o código sintetizado apresente erro de execução ou produza matriz discrepante, o ambiente isola o primeiro contraexemplo discordante, fornecendo ao modelo o índice da demonstração violada, a entrada (*input*), a saída esperada (*expected*) e a matriz efetivamente produzida (*produced*). O modelo é então estimulado a recalibrar sua solução a partir dessa discrepância. Esse processo iterativo estende-se até que o programa obtenha convergência integral em todos os exemplos de treino ou até o esgotamento do limite operacional de $K_{\max} = 5$ iterações.

2.3 Regras Anti-Trapça e Elicitação de Invariantes

Durante o recebimento de contraexemplos, constata-se com frequência que os modelos de linguagem recorrem a estratégias de sobreajuste (*overfitting*) superficial. Em vez de reformular a lógica geradora global, é comum a inserção de ramificações condicionais ad-hoc (*if/else*) calibradas exclusivamente para satisfazer o exemplo discordante recém-apresentado.

Para mitigar essa memorização espúria, a variante CEGIS Anti-Trapça (ANTI_CHEAT_RULES) interdita expressamente ramificações atreladas a coordenadas ou índices pontuais e impõe a elicitación prévia do invariante conceitual em linguagem natural antes da geração do código.

3 Metodologia e Fundamentação Estatística

Para avaliar formalmente a superioridade entre regimes de prompting frente à variabilidade estocástica inerente aos LLMs, adota-se um delineamento de inferência pareada não-paramétrica.

3.1 Formulação das Hipóteses

Sejam M_{base} e M_{var} duas estratégias de prompting sob comparação. Para cada tarefa $i \in \{1, \dots, N\}$, a variável $X_i \in \{0, 1\}$ denota o sucesso estrito (acerto conjunto em treino e no teste cego). Formulam-se dois testes de hipóteses:

1. Diferença Geral ($H_{0,\text{dif}}$):

$$H_{0,\text{dif}} : P(X_i^{(M_{\text{var}})} = 1) = P(X_i^{(M_{\text{base}})} = 1) \quad (1)$$

com hipótese alternativa bilateral $H_{1,\text{dif}} (P(X_i^{(M_{\text{var}})} = 1) \neq P(X_i^{(M_{\text{base}})} = 1))$.

2. Superioridade Direcional ($H_{0,\text{melhor}}$):

$$H_{0,\text{melhor}} : P(X_i^{(M_{\text{var}})} = 1) \leq P(X_i^{(M_{\text{base}})} = 1) \quad (2)$$

com hipótese alternativa unilateral $H_{1,\text{melhor}} (\Delta > 0)$.

3.2 Desenho Pareado e Tabela de Contingência

Dado que ambos os métodos avaliam o mesmo conjunto de tarefas, os desfechos são estruturados em uma tabela de contingência 2×2 , na qual $a = n_{11}$ denota acerto simultâneo, $b = n_{10}$ corresponde a regressões (acerto exclusivo do *baseline*), $c = n_{01}$ indica recuperações (acerto exclusivo da variante) e $d = n_{00}$ denota falha conjunta.

3.3 Teste Exato de McNemar

Para o teste de $H_{0,\text{dif}}$, aplica-se o teste exato de McNemar (McNemar 1947). Sob a hipótese nula, a contagem de pares discordantes ($n_{\text{disc}} = b + c$) segue uma distribuição Binomial($b + c, 0.5$), obtendo-se o p -valor bilateral exato por:

$$p_{\text{McNemar}} = \min \left(1, 2 \sum_{k=0}^{\min(b,c)} \binom{b+c}{k} \left(\frac{1}{2} \right)^{b+c} \right) \quad (3)$$

A formulação exata é indispensável no benchmark ARC, onde a frequência reduzida de pares discordantes compromete a aproximação assintótica via χ^2 .

3.4 Inferência por Bootstrap Pareado

Enquanto o teste de McNemar avalia a igualdade marginal, o método de bootstrap pareado com $B = 10.000$ reamostragens com reposição (Efron and Tibshirani 1993) viabiliza a inferência direcional sobre $H_{0,\text{melhor}}$ e a quantificação do tamanho do efeito. Em cada iteração j :

$$\Delta_j^* = \frac{1}{N} \sum_{i=1}^N \left(X_{i,j}^{*(M_{\text{var}})} - X_{i,j}^{*(M_{\text{base}})} \right) \quad (4)$$

A partir da diferença observada $\hat{\Delta} = (c - b)/N$, centralizam-se as réplicas sob a hipótese nula ($\tilde{\Delta}_j^* = \Delta_j^* - \hat{\Delta}$), computando-se o p -valor empírico unilateral por:

$$p_{\text{boot}} = \frac{1 + \sum_{j=1}^B \mathbb{I}(\tilde{\Delta}_j^* \geq \hat{\Delta})}{B + 1} \quad (5)$$

onde $\mathbb{I}(\cdot)$ representa a função indicadora. O intervalo de confiança de 95% para Δ é delimitado pelos percentis 2.5% e 97.5% da distribuição empírica de Δ^* .

3.5 Taxonomia dos Modos de Falha

Para dissecar a dinâmica microscópica da síntese, discriminam-se quatro modos operacionais: (i) **Recuperação Semântica** ($0 \rightarrow 1$), quando falhas do *baseline* são corrigidas após o ciclo CEGIS; (ii) **Regressão** ($1 \rightarrow 0$), quando soluções válidas são degradadas pelo refinamento; (iii) **Sobreajuste Espúrio / Falsa Convergência**, caracterizado por acerto pleno no treino com subsequente falha no teste cego; e (iv) **Teto de Representação**, correspondente à impossibilidade de convergir no treino dentro de $K_{\max}$ iterações.

4 Protocolo Experimental e Execução

A infraestrutura operacional foi estruturada com foco em reprodutibilidade estrita, isolamento de execução e controle das cotas de requisição das APIs.

Table 1: Resultados experimentais consolidados nos 100 problemas avaliados do ARC-AGI-1 sob as abordagens 1-Shot, CEGIS e CEGIS Anti-Trapça.

Modelo	Tarefas	Baseline	CEGIS	Anti-Trapça	Δ	Resolvidos	Req./Tarefa	Recuperação	Regressão
Grok 4.6	100	88.0%	92.0%	92.0%	+4.0%	92/100	1.13	4	0
DeepSeek V4 Flash 0731	100	74.0%	86.0%	86.0%	+12.0%	86/100	1.25	12	0
GPT-5.6 Luna	100	58.0%	77.0%	79.0%	+21.0%	79/100	2.00	19	0
Claude Sonnet 5	100	32.0%	62.0%	60.0%	+28.0%	60/100	2.90	30	0
Gemini 3.5 Flash Lite	100	21.0%	43.0%	45.0%	+24.0%	45/100	3.23	22	0
Gemini 3.1 Flash Lite	100	12.0%	31.0%	24.0%	+12.0%	24/100	4.07	19	0
GLM 5.3 Flash	100	10.0%	19.0%	17.0%	+7.0%	17/100	4.42	9	0
Leanstral 1.5.1	100	3.0%	7.0%	9.0%	+6.0%	9/100	4.69	4	0
GPT-5.4 Nano	100	1.0%	5.0%	3.0%	+2.0%	3/100	4.89	4	0

4.1 Critério de Avaliação e Sandbox

A execução dos códigos gerados ocorre em ambiente de sandbox hermético, sob restrição temporal estrita (`TIMEOUT_SECONDS = 2.0s`). O critério de sucesso é binário e exato: a função `transform(grid)` deve produzir matrizes dimensional e cromaticamente idênticas às de referência em todas as demonstrações de treino e na grade de teste cego.

4.2 Modelos e Infraestrutura de Execução

O pipeline oferece suporte transparente a modelos comerciais proprietários e de pesos abertos. Para compor o painel avaliativo, foram selecionados nove modelos representativos distribuídos em três faixas operacionais: A seleção abrange modelos de fronteira (Grok 4.6, DeepSeek V4 Flash 0731 e GPT-5.6 Luna), arquiteturas intermediárias (Claude Sonnet 5, Gemini 3.5 Flash Lite e Gemini 3.1 Flash Lite) e redes compactas de baixa latência (GLM 5.3 Flash, Leanstral 1.5.1 e GPT-5.4 Nano).

O subsistema de execução integra mecanismos de estabilidade:

- Controle adaptativo de taxa de requisições por minuto (RPM) e tokens por minuto (TPM);
- Monitoramento contínuo de cotas diárias (RPD);
- Persistência incremental (*Checkpoint & Resume*) com salvamento de estado após a conclusão de cada tarefa.

5 Resultados Experimentais e Análise

A avaliação empírica foi conduzida sobre uma amostra padronizada de 100 tarefas do conjunto de treino do ARC-AGI-1. A Tabela 1 consolida os resultados de acurácia e esforço computacional sob cada regime de prompting, enquanto a Tabela 2 sintetiza a validação inferencial pareada e a distribuição dos modos de falha.

5.1 Eficácia Geral do CEGIS e Monotonicidade

Os dados da Tabela 1 revelam que a síntese indutiva balizada por contraexemplos semânticos superou o *baseline* 1-Shot em todas as nove arquiteturas avaliadas. O acréscimo absoluto de acurácia (Δ) variou entre +2,0% e +30,0%. Esse avanço manifestou-se com particular intensidade em modelos intermediários: o Claude Sonnet 5 praticamente

duplicou sua taxa de acerto, saltando de 32,0% para 62,0% ($\Delta = +30,0\%$), ao passo que o Gemini 3.5 Flash Lite avançou de 21,0% para 45,0% ($\Delta = +24,0\%$).

Destaca-se um achado central de alta relevância prática: a estrita **monotonicidade** do processo de refinamento. Em todas as 900 avaliações pareadas registradas, o número de regressões ($1 \rightarrow 0$) foi rigorosamente **zero**. Enquanto pipelines heurísticos de auto-correção desestruturada frequentemente incorrem em deriva semântica catastrófica (destruindo soluções corretas ao tentar recalibrar detalhes), o contraexemplo minimal ($X, Y_{\text{expected}}, \hat{Y}_{\text{actual}}$) forneceu uma âncora corretiva que permitiu sanar incongruências locais preservando a integridade do algoritmo global.

Em termos de eficiência computacional, observou-se uma relação inversa entre o consumo médio de requisições por problema e a capacidade do modelo. Modelos de alta acurácia demandaram um custo iterativo marginal: o Grok 4.6 alcançou 92,0% de acurácia consumindo apenas 1,13 requisições/tarefa, e o DeepSeek V4 Flash 0731 atingiu 86,0% com 1,25 requisições/tarefa. Em contraste, arquiteturas com menor repertório de primitivas (GPT-5.4 Nano e Leanstral 1.5.1) esgotaram reiteradamente o orçamento de $K_{\text{max}} = 5$ iterações em problemas complexos, resultando em médias de 4,89 e 4,69 requisições/tarefa.

5.2 Inferência e Validação Estatística Pareada

A análise inferencial na Tabela 2 corrobora a solidez dos ganhos empíricos. A partir dos testes de permutação pareada e bootstrap ($B = 10.000$ réplicas), rejeitam-se as hipóteses de ausência de diferença ($H_{0,\text{dif}}$) e não-superioridade ($H_{0,\text{melhor}}$) ao nível $\alpha = 0,05$ em todos os modelos. Para redes de porte intermediário e grande (Claude Sonnet 5, Gemini 3.5 Flash Lite, GPT-5.6 Luna, Gemini 3.1 Flash Lite e DeepSeek V4 Flash 0731), os p -valores situaram-se estritamente abaixo de 0,001, com intervalos de confiança de 95% positivos (ex.: [21,0%, 39,0%] para Claude Sonnet 5 e [14,0%, 30,0%] para Gemini 3.5 Flash Lite), comprovando que os ganhos independem de flutuações amostrais.

Mesmo sob acentuada saturação, regime em que o Grok 4.6 atingia 88,0% no *baseline*, a recuperação de 4 tarefas adicionais alcançou significância estatística ($p_{\text{boot}} = 0,0431 < 0,05$, IC [1,0%, 8,0%]). Nos modelos compactos

Table 2: Testes de significância estatística pareada (permutação e Bootstrap, $B = 10.000$) e decomposição de erros sob feedback semântico ($\alpha = 0,05$).

Modelo	Δ_{CEGIS}	p_{perm} (CEGIS)	p_{boot} (CEGIS)	IC 95% Bootstrap	Sobreajuste Espúrio	Teto Repres.
Grok 4.6	+4.0%	0.0603	0.0431	[1.0%, 8.0%]	8	0
DeepSeek V4 Flash 0731	+12.0%	0.0005	0.0007	[6.0%, 19.0%]	10	4
GPT-5.6 Luna	+19.0%	0.0001	0.0001	[12.0%, 27.0%]	10	13
Claude Sonnet 5	+30.0%	0.0001	0.0001	[21.0%, 39.0%]	7	31
Gemini 3.5 Flash Lite	+22.0%	0.0001	0.0001	[14.0%, 30.0%]	10	47
Gemini 3.1 Flash Lite	+19.0%	0.0001	0.0001	[12.0%, 27.0%]	7	62
GLM 5.3 Flash	+9.0%	0.0012	0.0052	[4.0%, 15.0%]	0	81
Leanstral 1.5.1	+4.0%	0.0603	0.0457	[1.0%, 8.0%]	0	93
GPT-5.4 Nano	+4.0%	0.0603	0.0496	[1.0%, 8.0%]	0	95

(GLM 5.3 Flash, Leanstral 1.5.1 e GPT-5.4 Nano), o procedimento também indicou superioridade significativa ($p_{\text{boot}} < 0,05$), atestando a utilidade do CEGIS mesmo perante espaços representacionais restritos.

5.3 Decomposição de Falhas: Sobreajuste Espúrio vs. Teto

A análise dos modos de falha discrimina dois fenômenos conceituais opostos. A **recuperação semântica** ($0 \rightarrow 1$) constituiu o vetor dominante de aprimoramento, alcançando 30 tarefas no Claude Sonnet 5 e 22 no Gemini 3.5 Flash Lite. O contraexemplo isolado sanou primordialmente desvios de indexação matricial e inversão de cores em tarefas nas quais o modelo já intuía a topologia correta porém incorria em descalibrações pontuais.

Em contrapartida, o **sobreajuste espúrio** incidiu quase que exclusivamente sobre modelos de alta capacidade (10 tarefas em DeepSeek V4, GPT-5.6 e Gemini 3.5; 8 em Grok 4.6; 7 em Claude 5 e Gemini 3.1). Nesses casos, o código convergiu em 100% dos dados de treino mas falhou no teste cego, acomodando o contraexemplo via ramificações particulares sem apreender o invariante global. Por outro lado, em arquiteturas compactas prevaleceu o **teto de representação** (GPT-5.4 Nano: 95 tarefas; Leanstral: 93; GLM: 81), nas quais a convergência sequer foi obtida em treino após 5 iterações, evidenciando carência de primitivas lógico-geométricas no espaço de busca.

5.4 Efeito das Regras Anti-Trapça

A estratégia CEGIS com regras Anti-Trapça (ANTI_CHEAT_RULES), concebida para interditar ramificações artificiais e induzir a explicitação do invariante em linguagem natural, exerceu papel determinante em modelos intermediários. No Gemini 3.5 Flash Lite, promoveu ganho adicional de acurácia de 43,0% para 45,0% ($p_{\text{boot}} = 0,0001$, IC [16,0%, 33,0%]), e no GPT-5.6 Luna avançou de 77,0% para 79,0% (IC [13,0%, 30,0%]). O estímulo à reflexão conceitual prévia atenuou a propensão dos modelos a inserirem remendos parciais. Por sua vez, em modelos de fronteira (Grok 4.6 e DeepSeek V4 Flash), a acurácia manteve-se estável com ligeira economia no número de iterações demandadas.

6 Discussão e Escalonamento Empírico

A introdução de ciclos de refinamento orientados por execução redefine o compromisso entre busca simbólica e inferência neural. Ao fornecer contraexemplos detalhados em vez de incentivar a reescrita desestruturada do código, o pipeline CEGIS converte o LLM em um amostrador de hipóteses guiado por verificação formal, garantindo que nenhum acerto prévio seja corrompido (zero regressões).

O protocolo estatístico aqui formalizado preenche uma lacuna metodológica recorrente na literatura do ARC-AGI, na qual variações marginais de prompt costumam ser reportadas sem controle de significância pareada ou intervalos de confiança. Conforme demonstrado, a combinação do teste exato de McNemar com o bootstrap pareado permite discernir ganhos reais mesmo em amostras de 100 tarefas, estabelecendo um padrão de análise diretamente escalável para o conjunto integral do benchmark (800 tarefas de treino e avaliação).

7 Conclusão

Este trabalho estabelece uma metodologia formal e experimentalmente fundamentada para avaliar variações de prompting e síntese indutiva no ARC-AGI-1. Os resultados comprovam que a retroalimentação por contraexemplos semânticos (CEGIS) supera expressivamente a síntese direta 1-Shot em toda a escala de modelos, preservando estrita monotonicidade e viabilizando saltos de acurácia de até 30 pontos percentuais. A taxonomia de modos de falha articulada com inferência pareada fornece um arcabouço rigoroso, transparente e reproduzível para a evolução da síntese de programas com modelos neurais de linguagem.

References

- Chollet, F. 2019. On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547*.
- Efron, B.; and Tibshirani, R. J. 1993. *An Introduction to the Bootstrap*. Chapman and Hall/CRC.
- McNemar, Q. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2): 153–157.
- Solar-Lezama, A. 2008. *Program Synthesis by Sketching*. Ph.D. thesis, University of California, Berkeley.